Assignment 2 Deep Learning

Elena Petridou, Eleonora Roncaglia, Bashier Gulzar

November 2025

1 Abstract

2 Task 1: Generative Models

2.1 What are generative models?

Generative models are a family of models that do not only extract and simplify complex patterns from data, but are also then able to use this information to produce new outputs, related to the original data. In general, to achieve this, a model must first use *encoder layers*, which glean information from data, and steadily reduce the size of the structures which express the information, until the features can be contained in a compact, abstract representation. The process the encoder layers perform is called *downsampling*, and the compact, abstract feature space it produces is called the *latent space representation* of the original inputs. Subsequently, the model makes use of *decoder layers* which use the information in the Latent Space to output new samples which approximate the data that the model was fed in the beginning. This latter process is called *upsampling*. It is important that the latent feature space is somehow smaller and sparser than the original features in the dataset, as it "forces" the model to learn patterns about the data without taking the shortcut of simply regurgitating the exact inputs during upsampling.

In this assignment, we are asked to inspect two model architectures: Auto-Encoders (through a Convolutional Auto-Encoder (CAE) and a Variational Auto-Encoder (VAE)), and Generative Adversarial Networks (GANs), understand their workings, similarities and differences. The task at hand is an unsupervised learning task, whereby we are asked to locate a new dataset and using the provided architecture on our new dataset, create a visualisation which experiments with the Latent Feature Space our model extracts from the images in the dataset.

2.2 Auto-Encoders

2.2.1 Convolutional Auto-Encoder (CAE)

As a small recap, image data contains features which are not straight-forwardly interpretable (as opposed to tabular data, for example). This means that feature-extraction must be carried out in order to condense the information in the images into something that the model can use. As we have seen in Assignment 1, Convolutional Neural Networks (CNNs) do very well at this task by mimicking the visual cortex of humans, whose neurons do not process the whole image at once, but instead perceive images by breaking them down into local receptive fields and then spotting patterns within those fields. The first layer essentially does just this: splits the input image into perceptive fields, each of which is only assigned to a certain subset of its neurons. Each subsequent layer continues this process, reducing the size of the feature space with each iteration by combining these low-level, more "local" features into higher-level "global" features, from which the whole image can be approximated (Géron, 2022, p.448). This form of downsampling allows CNNs to make use of the spatial nature of images, which is what makes them so powerful. The final latent space representation of the images is a set of vectors.

For a CAE, it should be no surprise that Convolutional Layers make up the *encoder* portion of its architecture. The fact that the latent feature space created by the encoder of the network has lower dimensionality than the original data makes the autoencoder model *undercomplete* (Géron, 2022, p.570), which means that the model has no "trivial" way of re-producing the inputs it was given. It must try to re-create them using this latent vector space.

The *decoder* portion of the network is where the model tries to interpret the information contained in the *latent space representation* (or sometimes called "bottleneck") produced by the encoder, in order to produce outputs which resemble the given inputs as closely as possible. To do this, the decoder is made up of transposed convolutional layers, which up-sample and re-shape the layers contained in the latent dimensional space. Since not all of the original information learned is present in these abstracted layers, the model must do some "guessing", which is why constructing the output images is non-trivial, as this guessing is not random, but based on the learned patterns contained within the latent space.

Finally, the two portions of the network are connected to each other via a sequential layer which takes as inputs the encoder and decoder. In the code provided, this final model is compiled and optimized using the Adam optimizer and the binary cross-entropy loss. In encoder-decoder models, the difference the loss is minimizing is between the input images and the reconstructed images. In this case, for example, our CAE minimizes the binary cross-entropy loss between the pixel values in the inputs and the pixel values in the outputs (the re-constructions of the original images).

However, CAEs are not true generative models! The final output is only an as-faithful-as-possible reconstruction of the input images the model is trained on, and not truly new instances. The reason for this is that the decoder can only take in the exact vector representation of one of the input images and use it to re-construct that exact image.

In order for a model to be generative, we need to have the option of passing a vector which has been somehow modified to its decoder portion, and receive a new generated sample which closely resembles the input dataset but is not identical to any one of the images contained within it.

2.3 Variational Auto-Encoder (VAE)

2.3.1 Theoretical Background on VAE

As opposed to the CAEs discussed above, Variational Auto-Encoders (VAEs), **are** generative networks, and they achieve this by "solving" the aforementioned issue of the latent feature space being an unstructured set of vectors representing the input images.

To see what we mean by this, let us walk through a theoretical image generation process as described in Kingma and Welling (2013)'s seminal paper which introduced Variational Auto Encoders. Assume that an image X (which is made up of features, say x), comes from a "real world" process involving an "unobserved continuous random variable z" (Kingma & Welling, 2013, p. 2), through a 2-step process:

- A value of the latent $z^{(i)}$ is first generated from a prior distribution $p_{\theta*}(z)$
- The feature value for an image $x^{(i)}$ is then generated from some conditional distribution $p_{\theta*}(x|z)$

Where $\theta*$ are the true parameters of these distributions. Now, if we knew the exact prior distribution $p_{\theta*}(z)$ and the exact conditional distribution $p_{\theta*}(x|z)$ which produced the features of an image X, then creating a new image, say $X*$ which is somehow "similar" to X is simple. We could sample $z*$ from the prior distribution $p_{\theta*}(z)$. This $z*$ comes from the same distribution as z so it is necessarily similar to it. Then plug in to the conditional distribution $p_{\theta*}(x|z)$ to sample a set of $x*$ similar enough to the features of the original image. Then, the resulting image should be a "similar" but original image $X*$.

As in most applications, we do not know the true prior and conditional distributions which generated the features of our images. Thus, we can only try to approximate these. This is more or less what the simple gradient descent algorithm does, by trying different parameters θ and thus constructing an approximate posterior distribution $q_\theta(z|x)$. We call this the "recognition model". The problem for generating using this recognition model produced by such an approach is that there are no constraints on how this approximated probability function should behave; the only objective is that it reconstructs the original inputs as accurately as possible. For the goal of generation, it is useful to have a **well-behaved latent space**, which happens when the latent space possesses the properties of **continuity** and **completeness**. Continuity implies that points near each other on the distribution will be more or less similar meaning that latent features near each other will translate to related features on an image, and completeness implies that for any random value in the domain of the function, can yield meaningful latent features.

We can constrain the latent space such that it possesses continuity and completeness by modifying our training objective. Instead of simply using the reconstruction loss, we need to add a regularisation term to it, called the *Kullback-Leibler (KL) divergence*. The KL divergence term compares the dissimilarity of two probability distributions, so we can use this term to compare the learned distribution at each training step with a Normal distribution. As such, our new training objective involves a trade-off between the reconstruction loss and the KL-divergence term, meaning that the model's training objective is now to minimise reconstruction loss while getting the latent space as similar as possible to a Gaussian distribution. This whole term, is called the **Evidence Lower Bound (ELBO)**, given by:

$$ELBO(\theta, \phi) = \int q(z|\theta) log[p(x|z, \phi)]dz - D_{KL}[q(z|\theta)||p(z)]$$

Where the first term is the reconstruction loss of our constructed recognition distribution, and the second term is the KL divergence. The first term can also be approximated by sampling $z*$ from q, which then allows us to reformulate the ELBO as:

$$ELBO(\theta, \phi) \approx log[p(x|z^*, \phi)] - D_{KL}[q(z|\theta)||p(z)]$$

Here is where another crucial difference of VAEs pops up, the reparametrisation trick, which necessitates us looking at more or less what steps the VAE takes during training first, in order to understand why this is necessary.

Forward-Pass

- Encoder:
  1. Take in the image X and try to extract a feature set Z, just as in a CAE
  2. Unlike a CAE, instead of representing each feature z using a fixed numerical value, the model calculates a vector of means and a vector of (in our given example code a log-transformed) variances defining a Gaussian distribution $q(z|\theta)$ from which to randomly sample a latent feature vector.
  3. Randomly sample a point $z*$ from $q(z|\theta)$, and repeat this for all latent features in the image.
- Decoder:
  4. Take all the randomly sampled $z*$ and try to reconstruct an image using the latent variables therein through deconvolutional layers.

Here is where an essential challenge of training VAEs becomes evident: during backpropagation, moving from step 3 to step 2 is not possible, as we cannot move backwards through a random node – there is no deterministic way through which to "walk backwards" through step 3, necessitating an adjustment to allow for the gradients to flow properly. This adjustment is called the *reparametrisation trick* and changes step 3 such that it now is:

REPARAMETRISATION TRICK: Sample error term ϵ from a standard normal distribution and compute the values of z^* as: $z^* = \mu + \sigma * \epsilon$

Using the reparametrisation trick, there is now a deterministic solution to step 3, allowing the gradients to freely flow backwards as well. In our application, the ELBO is minimised using the Adam optimiser.

2.3.2 VAE in practice: tuning the model

In order to explore the generative capabilities of the VAE and GAN, we looked for datasets that have a degree of variability large enough to make the task interesting, but small enough to make new image generation viable and the new samples recognisable as a plausible image. To this end, we used a playing cards dataset sourced via the website Roboflow. It contained a total of 8152 images when directly downloaded, from which we filtered out images of playing cards which had a dark background (the card itself was a colour other than white), as that would introduce too much heterogeneity and thus hinder our model's generative capabilities. The resulting filtered dataset contained 6166 images, a sample grid of which is shown in figure 1

Figure 1: A grid of nine sample images from the playing cards set we used to train our generative models

Using the code provided as part of the assignment, we began an iterative training process where we tried tuning different parameters and observed the effects they had on the generated images.

Latent Dimensionality The first parameter that we experimented with was changing the dimensionality of the latent space. The default latent dimension size for the faces dataset was set to 64, which for the cards dataset held the model back from creating full symbols on the cards. This makes sense, as the latent space dimensionality determines how "detailed" the latent representation of our data will be. A larger latent space implies that the model tries to retain too many details about the images, which it appears in the case of playing cards leads to it confusing noise (differences in the placement of symbols on the same type of card in different decks, for example) as signal. Thus, reducing the latent space to a size of **16** led to both the VAE and GAN generating images that were more plausible as playing cards.

Loss re-balancing We also noticed when training our VAE that even during early epochs, all nine of the generated cards contained in the sample grid would look too uniform, with only faint markings on each card. We know the latent space of a VAE is constructed via the trade-off between the reconstruction and KL losses, with the former focusing on creating accurate recreations of the images and the latter organising the distribution of the latent space such that its divergence from a normal distribution is minimal. As such, we concluded that the images looking too uniform and without particular features standing out would correspond with the KL loss dominating training (early). This implies that the model does not get to first learn the unique features of the cards and then organise the latent space, but instead prioritises organising the latent space too soon in the training process, causing the generated samples to be too homogeneous to resemble any real card.

To combat this, we decided to re-organise the way the VAE is fitted using a custom function such that KL loss could be penalised in earlier training epochs, such that the reconstruction loss prevails at the outset. The maximum penalisation weight the KL loss can receive is **0.6**, which is increased linearly over 15 epochs ("the warm-up" epochs). Thus, we start training with the model only trying to optimize the reconstruction loss during the first epoch, and as epochs increase, it slowly incorporates more of the KL-divergence loss such that the KL loss makes up 0.6 of the total ELBO by epoch 15.

Batch Normalisation In order to further optimize the model to our dataset, we implemented some changes to the architecture by writing a custom `build_vae` function which implements its own encoder and decoder architecture. The key differences from the provided code are that following each Convolutional layer in both the encoder and decoder we implemented Batch Normalisation. As we found out in Assignment 1, Batch Normalisation can make training more stable by essentially ensuring that there is less variation in the training inputs between training batches – a very important property in our problem, as our dataset is quite heterogeneous (with cards having different background and symbol colours, not the same orientation, etc.).

Additional callbacks After a small number of epochs, we noticed that our model was having trouble improving the generated images, leading us to believe that perhaps the learning rate was too high and causing the optimiser to be stuck in a local minimum of the gradient. As such, we implemented a custom callback provided to our fit function which reduces the learning rate by a factor of 0.5 if for 3 epochs the loss does not improve by 0.001. Another callback we implemented was early stopping in case the loss did not decrease by 0.0001 within 3 epochs, but this only kicks in after 20 training epochs.

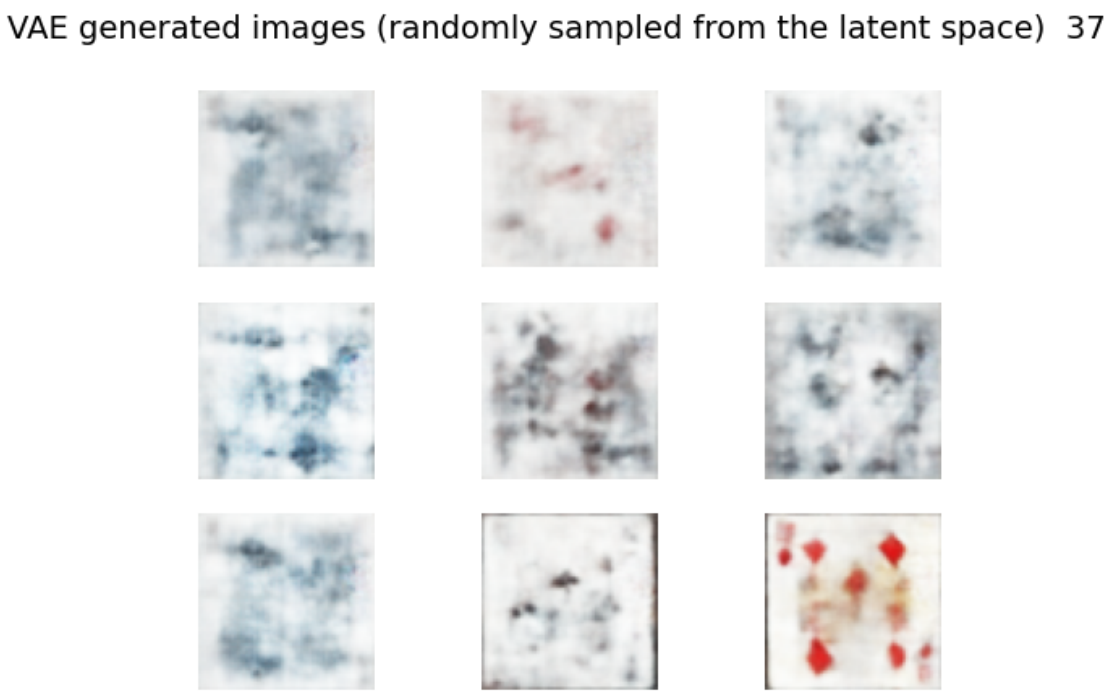


Figure 2: Sample grid of generated images produced by the VAE after training for 37 epochs. The outputs are not clear, and the produced images also varied dramatically per epoch.

Leaky ReLU Another major change in our new VAE-building function is the use of the Leaky ReLU activation function between the convolutional layers for the encoder and the decoder. This decision was motivated by the fact that after certain training epochs, the improvement in the images output by our model was not significant. After attempting to adjust the learning rate and loss weights (which is discussed in the sub-section 2.3.2), we came across the "dead neuron problem" which can occur when using the ReLU activation function (Ramadhan, 2021). Essentially, ReLU can at some point during training assign such a negative weight to a certain neuron, that this neuron "shuts off" for the rest of the training epochs, outputting zero no matter what the input. This can essentially cause some of the neurons in our network to drop out. While ideally we would have liked to investigate whether this was the case, given the fact that we tried adjusting the weighting of the loss functions and the learning rate, as well as changing optimizers but still could not get this "learning plateau" to go away without this intervention, we think that switching to the Leaky ReLU as the activation function for our custom-written encoder and decoder for the VAE is a reasonable choice, as this change allowed the model to overcome the stagnation.

Coefficient We also modified the coefficient for the random samples from the latent space that we use in order to produce images. The coefficient corresponds to the standard deviation of the sampled vectors, and we found that a smaller coefficient (more similar vectors) yields better results. In terms of cards, larger coefficients cause the model to mix up visual features that belong on "character cards" - meaning King, Queen, Joker, etc. - with the visual features that belong on number cards - meaning repeated symbols -. In terms of the latent space, this happens because the model assumes that all cards come from the same underlying feature distribution. However, this is not true in the same way as for faces. We could conceptualise the real feature space of the images more accurately as a set of distributions. For example: one distribution describing the features of the "character cards" and one distribution describing the features of the number cards. Since our latent space is described by one distribution for all the cards therefore, the higher the standard deviation of the latent feature vectors that we upsample to produce the visualisations, the more likely we are to produce cards that attempt to bring together features which should not go on the same card: for example, a large symbol in the center of the card (a feature of only the character cards) along with smaller symbols surrounding it (a feature of only the number cards).

2.4 Generative Adversarial Network (GAN)

2.4.1 Theoretical Background on GAN

Generative Adversarial Networks employ a different training philosophy than auto-encoders which also makes their architecture very different. Instead of being made up of an encoder and a decoder which work "collaboratively" a GAN is made up of a **generator** and a **discriminator**, which are optimised for different tasks.

The **generator** takes in n-dimensional vectors (essentially vectors of random numbers) which are meant to "originate from" the latent feature space, and using deconvolutional layers tries to upsample these vectors into images. This is similar to what a decoder does. At the end, it feeds these constructed images to the discriminator.

The **discriminator** takes in a set of real images and a set of generated images provided to it by the generator. Then, it behaves like a binary classifier which uses Convolutional Neural Network architecture (convolutional layers) to construct a latent variable space for each of the two sets, and using the latent features, decides whether the real and constructed images come from the same underlying distribution. This means that the discriminator is essentially trying to decide whether it can accurately tell apart real and constructed images. At the end, the discriminator passes this feedback back to the generator.

The training process for each of these parts of the network depends on each others' outputs, and can be considered *adversarial* due to the fact that the two parts are attempting to "fool" each other. To explain how this adversarial relationship works, it helps to examine the loss functions for each of the parts during training. The loss function for GANs is split into two parts. The

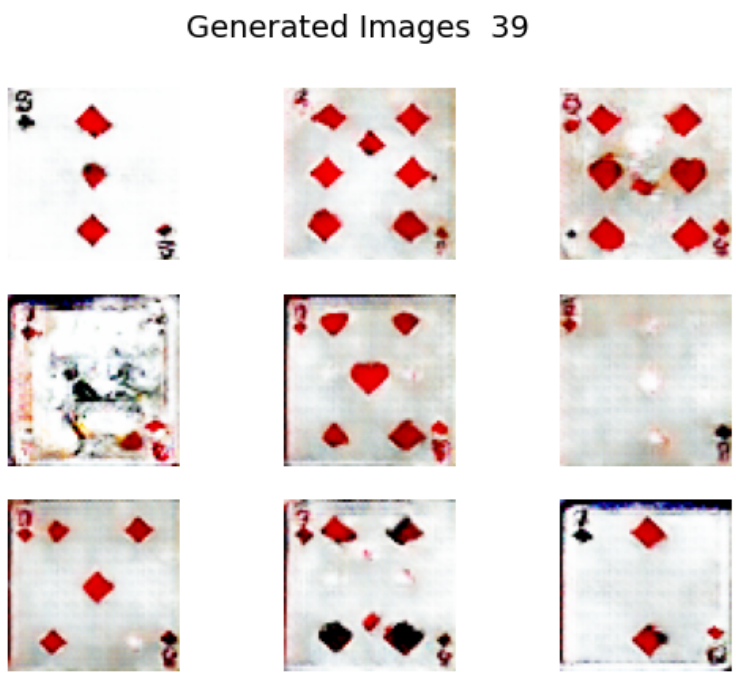


Figure 3: A sample grid of the generated images from our trained GAN, on epoch 39. We see that the outputs look much more like plausible playing cards than those produced by the VAE.

discriminator's (theoretical) loss is given by (Pelt, 2025, p. 37):

$$\hat{\phi} = argmin_{\phi}\Big[\sum_{j} -log\Big(1 - sigmoid\big(f(x_j^*, \phi)\big)\Big) - \sum_{i} log\Big(sigmoid\big(f(x_i, \phi)\big)\Big)\Big] \tag{1}$$

And the generator's (theoretical) loss is given by (Pelt, 2025, p. 37):

$$\hat{\theta} = argmax_{\theta}\Big[min_{\phi}\Big[\sum_{j} -log\Big(1 - sigmoid\big(f(x_j^*, \phi)\big)\Big) - \sum_{i} log\Big(sigmoid\big(f(x_i, \phi)\big)\Big)\Big]\Big] \tag{2}$$

In particular, we notice here the discriminator must minimise the function for ϕ, whereas for the generator, the whole expression must be maximised for θ. Essentially, the parameters for the generator θ are selected in such a way that they maximise the extent to which the discriminator is unable to tell the generated apart from the real samples.

A GAN is similar to a VAE in that it is also a **latent variable model** and aims to have a **well-behaved latent space**. Its way of generating new images is also, as for the VAE, to try and sample a set of new images $x*$ which came from the same distribution as our training images x. (Pelt, 2025), but the way in which it constructs an approximation of this "real image distribution" is different to how a VAE does it. As we have seen in the previous section, the VAE constricts the latent space via the KL-divergence loss which forces it to follow a normal distribution. On the flip side, GANs begin by postulating a random latent space (by randomly sampling vectors from a normal distribution in our application, for example) and then "shape" it through the adversarial training procedure such that it more closely approximates the distribution of the real images. This happens due to the fact that as the generator changes the latent variables' distribution, it is "punished" by the discriminator if this new distribution does not closely approximate that of the real images. Importantly, unlike a VAE, a GAN does not try to postulate a simple prior distribution for the latent space. In practice this means that its latent space is not smooth and continuous like the VAE's.

2.4.2 GAN in practice: tuning the model

Since the predictions with the default training architecture were promising, we did not define a custom function for fitting GANs. Instead, we specified some parameters based on what we found out about the nature of our task from tuning the VAE. Specifically, we specified a small latent feature dimension of 16, as before.

Training stopped at epoch 39, and produced the sample output grid in Figure

2.5 Exploring the latent feature space of each model via linear interpolation

2.5.1 What is latent space interpolation?

The latent feature space produced by generative models can be conceptualised in an abstract way but is difficult to explore in a more practical way. One way of visualising how the latent space of the model is organised is by using *latent space interpolation*.

Linear latent space interpolation involves sampling two random vectors z_a and z_b from the latent space, and then using formula 3 to gradually "move" along the latent space from z_a to z_b. ("Interpolation and Traversal in Latent Space", 2025). Then, if we upsample the latent vector produced at each step on this path from z_a to z_b, we can see how an image changes as we linearly move through the constructed latent feature space.

$$z_{interp}(a) = (1 - a) * z_a + az_b \tag{3}$$

Checking the produced visualisations at each step can help us detect whether the latent space is **continuous** and **complete**. If the latent space possesses these properties, images should form a smooth transition from the initial image produced by upsampling z_a to the final image produced by upsampling z_b. Moreover, a well-behaved latent space should ideally produce changes to the images that make sense to the human eye.

2.5.2 Interpolating the latent space of the VAE

Interpolating the latent space of the VAE produced the visualisations shown in Figure 4

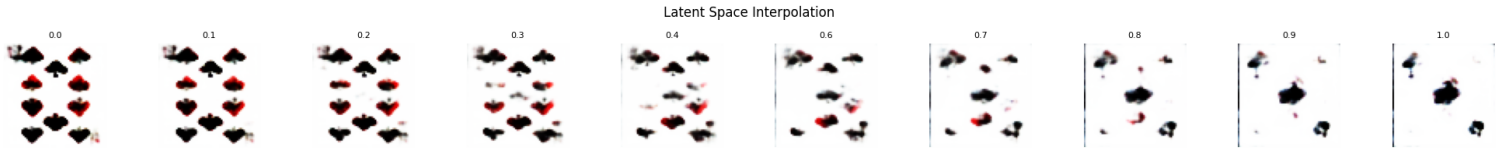


Figure 4: Linear interpolation of the latent space of the VAE. This visualisation shows an image turning from a generated sample of a 10 card to a generated sample of an ace card.

This visualisation shows that our first randomly sampled latent variable z_a upsamples to an image of a playing card with number 10. As we linearly move towards vector z_b, the number of symbols on the card gradually decreases. For example, once we have moved 70% of the way towards z_b, this intermediate sample contains approximately three symbols, two of which disappear more or less at the next "step". We finally end up with a card resembling an ace.

The line on which this move from z_a to z_b has taken place therefore, seems to be the latent dimension capturing whether a card is the number 10 or an ace. Unfortunately, as evidenced by the noise in our generated samples at every step, the latent space of our VAE is not completely well-behaved, as the model is "confused" about whether red colour should be included on the card, as well as which symbols should be placed where. Moreover, the outputs at each step do not represent a realistic generated sample of a real playing card, which would be the perfect outcome in a well-behaved latent feature space.

However, this visualisation shows that our latent space is more or less continuous, as the transition between the two cards moves in the "right direction" with every given step. By this we mean that as we move towards z_b, the number of symbols on the card gradually decreases. Another interesting observation about this visualisation is that the symbol appears to be the same between the cards generated by z_a and z_b. This means that to some extent, the latent feature space of our model can distinguish the difference between the number of symbols and the type of symbol which goes on a card.

2.5.3 Interpolating the latent space of the GAN

Finally, we produced some visualisations via linear latent space interpolation for our trained GAN.

In Figure 5 we see a linear interpolation of the latent feature space of the GAN, where the upsampled latent feature vector z_a corresponds to a generated image of a red three of diamonds. As we move towards the second sampled latent vector z_b, the card changes into a black three of spades. Thus, the "line" across the latent feature space we traverse between these two vectors seems to correspond to a change in symbol and change in colour for the same number card. Note that the card remains a three between the two random points, meaning that to some extent the model picks up on the fact that a card can have different colour and symbol variations while corresponding to the same number.

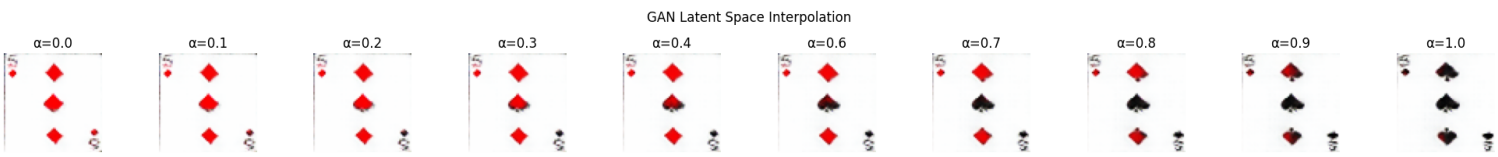


Figure 5: Linear latent space interpolation of the GAN model. This visualisation shows that the move from a three of diamonds to a three of spades involves changing the colour and the symbol of the card.

Another interesting visualisation from the latent feature space of the GAN is shown in Figure 6, where the initial generated card is a red two of hearts, which as we move through the latent space slowly turns into a red five of hearts. Throughout this whole traversal, the colour of the card does not change, onnly the number of hearts does: first disappearing shortly, but then increasing to four and ending up at five. What this visualisation shows is that the latent feature space can "conceptualise" the fact that cards exist in different numbers for the same colour and symbol combination, as the colour and symbol of the card do not change, only the number of symbols it contains.

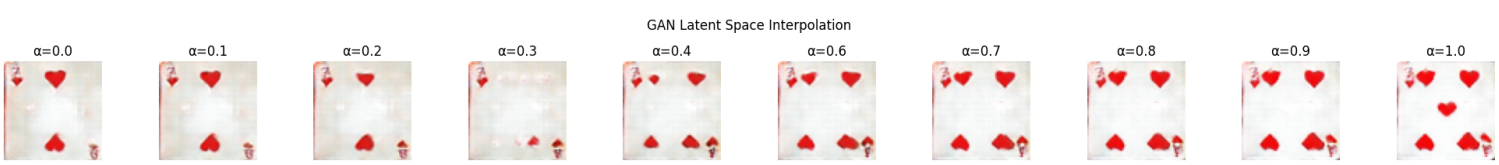


Figure 6: Another linear latent space interpolation of the GAN model. This visualisation shows a card going from a red two of hearts to a red five of hearts.

The visualisations of the GAN's latent feature space were much more promising than those of the VAE, despite the fact that we expended more effort in optimising the parameters for the VAE for our task. A possible explanation for this lies in how the two model differ in the way that they organise their latent feature space. As mentioned before, the VAE tries to ensure that its latent feature space conforms as closely as possible to a normal distribution – this is how it ensures that upsampled images from latent feature vectors that are closer together in this distribution will end up somehow being similar images. However, the fact that the VAE "pushes" the latent space's distribution towards an assumed normal prior means that it sometimes simplifies the patterns in the data, in favour of being faithful to the original training samples. With the GAN, the the discriminator simply penalises the generator when it generates images that are too outlandish, meaning that the model is allowed to learn an overall "rich similarity metric for images so as to discriminate them from non-images" (Larsen et al., 2016, p.2). In a dataset such as ours, where subtle differences in the placement or colour of a symbol completely changes which type of card an image represents, such a "smoothing"

effect caused by the VAE can have the result of -for instance- producing symbols that are not sharp enough to be recognisable (e.g.: the spades in Figure 4 not being easily distinguishable as spades instead of clovers).

3 Task 2: Recurrent Models

We are tasked with developing a set of models that learn to perform simple arithmetic operations: $+$ and $-$. Each model has to be able to deal with different data modalities. Specifically 3 models have to designed: text-to-text, image-to-text and text-to-image.

3.1 Workings of recurrent models

Whatever modality the input might be, text or image, the main property of the data is that the data can be described as a sequence of integers and operations. To accurately learn how to perform arithmetic the model has to be able to predict the outcome conditional on all previous inputs of this sequence of numbers and operators. Let the entire input equation be described as a sequence $\mathbf{x} = (x_t, x_{t-1}, ..., x_1)$, and each symbol (digit, operator, etc.) a token. Then, formally, the goal of modeling sequence data is to model the conditional distribution

$$P(x_t|x_{t-1}, ..., x_1).$$

The inevitable issue is that for each timestep in the sequence the data by which we estimate $P(x_t)$ increases. As an example, examine the conditional distributions $P(x_3|x_2, x_1)$ and $P(x_{10}|x_9, x_8, .., x_1)$. Pure recurrent neural networks solve this issue relying on h_{t-1}, the hidden state: a latent variable that encapsulates information that was learned in previous timesteps.

$$P(x_t|x_{t-1}, ..., x_1) \approx P(x_t|h_{t-1}),$$

$$h_t = f(x_t, h_{t-1}).$$

Let us consider a minibatch of training data $\mathbf{X}_t \in \mathbb{R}^{n\times d}$, in which n represents the amount of sequences in a batch and d the amount of features per timestep. Also consider $\mathbf{H}_t \in \mathbb{R}^{n\times h}$, in which h denotes the amount of hidden units in the hidden layer. Hidden layers without hidden states calculate their output $\mathbf{H}$ by $\mathbf{H} = \phi(\mathbf{X}\mathbf{W}_{xh} + \mathbf{b}_h)$, with ϕ as the activation function, $\mathbf{W}$ as the weight matrix with dimensions x: inputs; h: hidden nodes; and $\mathbf{b}$, the bias vector. The inclusion of timesteps reshapes the hidden layer calculation to be $\mathbf{H}_t = \phi(\mathbf{X}_t\mathbf{W}_{xh} + \mathbf{H}_{t-1}\mathbf{W}_{hh} + \mathbf{b}_h)$, (where t denotes the timestep), with the activation function now being either a tangh or sigmoid function. This changes the hidden layer output to be the sum of the input in the current time step and the hidden layer output of the previous step summarized into $\mathbf{H}_t$. This effectively circumvents the issue of growing data per timestep, as all information of previous timesteps is now described by the latent variable $\mathbf{H}_t$.

However, two issues arise with this structure:

1. h_t tries to encapsulate all information learned in previous steps universally, even if some timesteps carry less important information than others.
2. Usage of the tangh or sigmoid activation functions results in vanishing gradients. The outcome of taking each function's derivative strictly takes on values between 0 and 1. During backpropagation through time(steps) the partial derivatives of these functions are repeatedly multiplied with each other per timestep in a sequence, causing the gradient to shrink exponentially with the amount of steps in a given sequence.

3.2 Long short-term memory layers

To circumvent these issues, Long short-term memory (LSTM) layers are applied. These hidden layers once again employ the latent variable h_t but on top of that introduce a second latent variable c_t, the cell state. This variable effectively plays the role of the long term memory of the model; it governs what information should be forgotten, retained and used in calculation of h_t, by proxy influencing calculations of x_t. This functionality is achieved through gates: fully connected layers with sigmoid activation outputting values between 0 and 1. In particular 3 gates are used – the input, forget and output gate:

$$\mathbf{I}_t = \sigma(\mathbf{X}_t\mathbf{W}_{xi} + \mathbf{H}_{t-1}\mathbf{W}_{hi} + \mathbf{b}_i),$$

$$\mathbf{F}_t = \sigma(\mathbf{X}_t\mathbf{W}_{xf} + \mathbf{H}_{t-1}\mathbf{W}_{hf} + \mathbf{b}_f),$$

$$\mathbf{O}_t = \sigma(\mathbf{X}_t\mathbf{W}_{xo} + \mathbf{H}_{t-1}\mathbf{W}_{ho} + \mathbf{b}_o),$$

$$W_{xi}, W_{xf}, W_{xo} \in \mathbb{R}^{d\times h}, \qquad W_{hi}, W_{hf}, W_{ho} \in \mathbb{R}^{h\times h}, \qquad \mathbf{b}_i, \mathbf{b}_f, \mathbf{b}_o \in \mathbb{R}^{1\times h},$$

each $\mathbf{W}$ representing a weight matrix and $\mathbf{b}$ representing the biases of each function.

Per timestep the $\mathbf{C}_t$ gets updated in two steps. First, the amount of information retained from previous steps is adjusted by $\mathbf{F}_t \odot \mathbf{C}_{t-1}$. Next, the model has to decide how much information has to be let into the long term memory at the current step. For this the input gate is implemented. The issue is that the output of the input gate is a vector with values between 0 and 1, whereas the cell state is $c_t \in \mathbb{R}$. Thus the output vector has to be multiplied with the actual information that can be let in the memory. For this reason the candidate cell state or $\tilde{\mathbf{C}}_t$, is computed by

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{X}_t\mathbf{W}_{xc} + \mathbf{H}_{t-1}\mathbf{W}_{hc} + \mathbf{b}_c).$$

The candidate cell state serves as all total information that potentially can enter the long term memory, what information actually enters the cell state is calculated by $\mathbf{I}_t \odot \tilde{\mathbf{C}}_{t-1}$. This amounts to calculation of the cell state per time step as:

$$\mathbf{C}_t = \mathbf{F}_t \odot \mathbf{C}_{t-1} + \mathbf{I}_t \odot \tilde{\mathbf{C}}_t.$$

Finally, information from the cell state has to be passed on to the hidden state, governed by the output gate in the form of $\mathbf{H}_t = \mathbf{O}_t \odot tanh(\mathbf{C}_t)$, which is used to calculate the probability distribution $p(x_{t+1}|\mathbf{H}_t)$ in the same way as with vanilla RNNs. An important aspect is that the cell state by design is additive, whereas in plain RNN the only latent variable h_t is multiplicative. This in combination with the fact that the model can now tune how much it can forget of its long term memory strongly reduces the issue of vanishing gradients, as during backpropagation weight matrices are not multiplied with each other repeatedly.

3.3 Model architecture

3.3.1 Text to Text

As earlier specified, we are tasked with designing models that can handle different data modalities. We first tackle the text to text model as described by the assignment. The code for this model is already written, a general description of the model follows. Firstly, the labels are one-hot encoded so that the dimensionality of the inputs and outputs are set. Namely, 13 dimensions for all digits 0-9, the '+' and '-' operand, and '_' as padding. The model starts with an LSTM layer of 256 units, taking in a batch of sequences encoded in the 13 earlier specified dimensions. This layer effectively serves as the encoder of the input, encoding the input sequences of length 13 into the hidden state vector h_t of 256 scalars. This vector is passed on to the RepeatVector layer, which as the name suggests, repeats the vector by max_answer_length - 3 in our specific case - resulting in a matrix of size 3×256. The output is passed on to the decoder LSTM layer which learns the temporal dependencies by going over the sequence of the 3 repeated vectors and calculating h_t sequentially. Consequently, this layer has the $return_sequences = True$ flag, which outputs the hidden unit vector calculated on all timesteps instead of the last timestep i.e. the three updated hidden vectors. These updated hidden unit vectors are passed onto a TimeDistributed layer, which specifies that a certain operation has to be performed per timestep. That operation is application of a dense layer with a softmax activation function of length 13 - the amount of unique characters in the dataset - and predicts which token should be predicted for that specific hidden unit vector. Given that model predicts a probability distribution over discrete tokens for each timestep the loss function used to train the model is a categorical cross-entropy loss paired with the Adam optimizer.

3.3.2 Image to Text

The overall structure for the image to text model is the same; the only difference is the input for the model. The images have in some way been condensed to a vector that summarizes its content. To this end a convolutional neural network is applied to classify the digits. The setup for this network is as follows: First a convolutional layer with 32 filters is applied with a ReLu activation function and padding to learn global patterns per image in an input sequence. The output gets sent to a MaxPooling layer which extracts the highest value per 2×2 feature map. This setup of convolutional layer to MaxPooling layer is repeated one more time, but now with 64 filters to learn more loccalized features per image. The output is flattened into 1 vector, which on its turn is connected to a dense layer of 128 hidden units, effectively condensing the images into a vector of size 128. The CNN network is applied per timestep in the sequence through the TimeDistributed layer and its output is sent to the LSTM encoder layer, after which the entire process happens identical to the Text-to-Text network.

3.3.3 Text to Image

Once again most of the architecture stays the same. The input is once again text one-hot encoded into 13 dimensions and goes through the LSTM encoder $\rightarrow$ RepeatVector $\rightarrow$ LSTM decoder architecture. The difference lies in how the output of the LSTM decoder layer is used. This output, a matrix of 3×256 containing the calculated h_t latent vectors for each output, has to be upscaled to an image of $28 \times 28 \times 1$. For this an encoder network is designed which can be applied to each timestep in the LSTM decoder layer output by the TimeDistribution layer. The encoder takes a vector of size 256 and upscales the vector to size 12544 ($256 \times 7 \times 7$) by application of 3 dense layers of that size. The output of these layers is reshaped into a tensor of shape $7 \times 7 \times 256$, in which the first two values represent the width and height of the image and the last value the amount of channels of that image. After that a sequence of 5 deconvolutional layers are applied to shrink the channels back to 1 channel (grayscale) and simultaneously upscale the image to size 28×28, corresponding to the original image size. The first two layers maintain a stride parameter of 2, thereby uscapling the width and size from 7×7 to 28×28 in two steps. Simultaneously the image gets compressed from 258 to 1 channels over the span of these 5 deconvolutional layers by halving the filters per layer, i.e. halving the dimensions of the outputspace. The last deconvolutional layer not only compresses the previous input to 1 channel (grayscale) but also uses a sigmoid activation function to map all input values to be between 0 and 1, representing pixel intensity of the image. This network is applied to each latent vector outputted from the LSTM decoder layer, resulting in image prediction. Given that the output variable, y and $\hat{y}$ are now images a categorical cross-entropy loss function will not work. Thus for this model a binary cross-entropy loss function is used; each pixel value for the true y is compared with the predicted $\hat{y}$ and loss is applied based on the difference.

3.3.4 Encoder LSTM layers

Up until this point all models have been described by using one LSTM layer to encode the data. This layer learns temporal dependencies of a sequence on one abstraction level. Consequently, the final decoder layer can only use the information that this encoder layer encodes. The issue with this is that we rely on only one layer to capture all temporal dependencies and pass it in a perfectly summarized vector to the decoder LSTM layer. It might instead be a smarter option to try and enhance the encoder structure such that the decoder layer has better information to work with. A natural way to achieve this is to stack multiple LSTM encoder layers on top of each other, each LSTM encoder layer learning temporal dependencies on multiple abstraction levels. This could be especially useful when the input sequences are long, in which a distinction can be made over local timestep temporal dependencies (Ex: '1' and '0' making the integer 10) and global sequence temporal dependencies (Ex: the structure of 'digit','digit','operator','digit','digit'). To achieve this functionality one only needs to specify multiple LSTM layers with $return_sequences = True$, each layer returning all calculated hidden unit vectors per timestep, with only the final layer condensing all hidden unit vectors into one vector by setting $return_sequences$ to $False$. The merit of this becomes increasingly important with larger sequences. However our maximum input sequence length consists of 5 timesteps, the hypothesis is therefore that addition of these layers will not matter that much. It will surely improve the model in some way as the model has now more capacity to learn, but there might not be that much temporal dependency structures to pick up in general, possibly making the improvement negligible.

3.4 Results/discussion

3.4.1 Text-to-Text model

The goal of the first model is to train the model in such a way that it learns arithmetic instead of simply remembering the entire training set, to this end multiple training-test splits are trained (Figure 7). Naturally, the more training data there is (the lower the test split fraction) the better the model performs, reaching the highest validation accuracy of 0.97 with a train-test split of 0.2. Subsequent higher train-test splits achieving only equal or lower validation scores indicative that the model might be overfitting at this point.

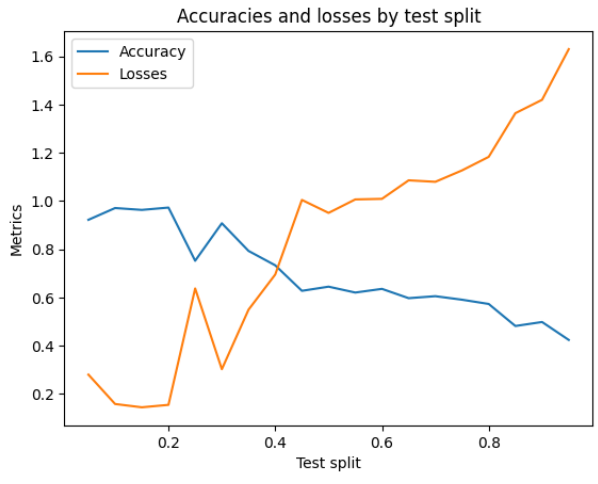


Figure 7: Loss and accuracy per Text-to-Text training split

To better view the errors the model makes, we examine a sample from the 0.5 train-test split (Table 1). Issues seem to appear at the exact precision of the calculations. While most of the predictions are wrong, the values are not off by much. Potential issues could be traced back to not completely learning all temporal dependencies. The model correctly learns what needs to happen per operator and in general guesses what the value should be, but there might be issues with forgetting local temporal dependencies, i.e. forgetting which numbers are next to each other resulting in small systematic errors. The model might be learning more superficial patterns than actual arithmetic.

Table 1: Text-to-Text model predictions and true value

Equation	Prediction	Correct Answer	Prediction deviation
53 + 25	82	78	+4
10 - 20	-1	-10	+9
43 + 34	86	77	+9
5 + 57	62	62	0
69 + 51	121	120	+1
59 - 81	-26	-22	-4
55 + 36	93	91	+2
15 + 1	23	16	+7
98 - 85	10	13	-3
40 - 57	-13	-17	+4
17 - 62	-40	-45	+5

3.4.2 Image-to-Text model

As seen in the previous section the Text-to-Text setup works well (whether it be through memorisation or actual understanding of arithmetic). In this Image-to-Text model, the only change is how the input is encoded. Whereas in the previous setting it was a sequence of one-hot coded vectors the sequence now consists of a series of images. The issue than becomes a matter of encoding the image as well as possible so that this summarisation of each image is nearly as good as the original one-hot encoded input sequence. Looking at the validation accuracies over different training splits of the model this seems to have been achieved successfully with the CNN encoder setup (Figure 8). The validation accuracy of this model on the 0.2 train-test splits achieves an accuracy of 0.90, smaller than the 0.97 test accuracies of the text-to-text model. Given that nothing was changed on the LSTM encoding $\rightarrow$ LSTM decoding structure it is safe to say that this difference is attributable to the difference in encoding. Apparently our CNN image encoder works well enough to summarize most numbers in a vector, but does falter at some points, leading to a drop in test accuracy.

3.4.3 Text-to-Image model

The text-to-image model is harder to evaluate as it outputs generated images rather than predicting text. There is no clear cut way to measure the quality of image generation, therefore we simply opt for measuring the performance of the model by clarity of the image and whether we think that the images predicted are valid results for the given equation (Figure 9). For the 0.2 train-test split the model consistently captures the right numbers and predicts most equations correctly, albeit a bit fuzzy. The latent space encoded by the hidden state and cell state correctly encapsulate the essential information needed to represent the predicted digit as an image. The output does remain a bit fuzzy, which can most likely be attributed to the binary cross-entropy loss function used. This loss function only compares each pixel value in the original 28×28 image input y with the predicted $\hat{y}$. This completely skips

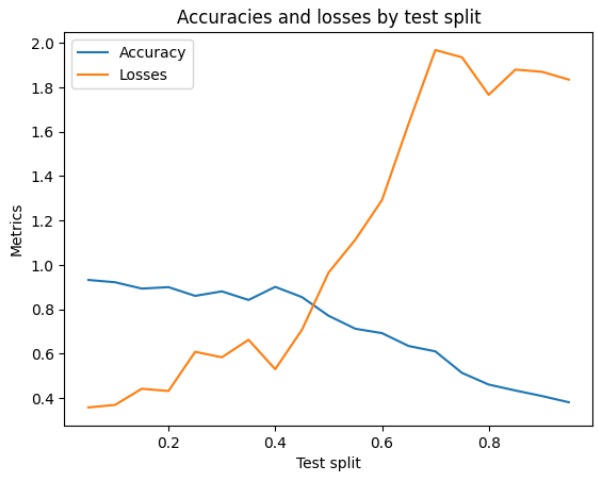
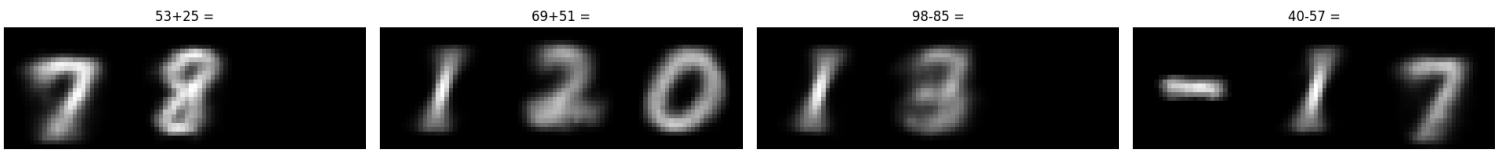


Figure 8: Loss and accuracy per Image-to-Text training split

possible edges or structure and instead forces the model to take an average over all different ways the digit was written, resulting in fuzzy numbers.



(a) (b) (c) (d)

Figure 9: Image prediction of equations

3.4.4 Usage of multiple LSTM encoder layers

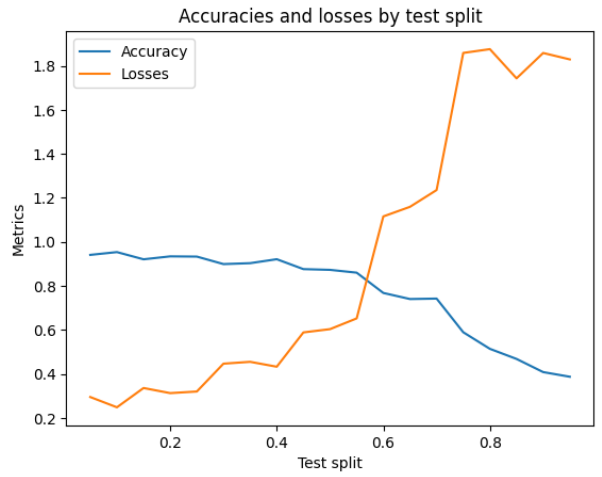
Text-to-Text As specified earlier it could be beneficial to use deeper encoding by application of an additional LSTM layer. Application of an additional LSTM layer to the text-to-text model seems to result in negligible improvements. On the 0.2 train-test split both models reach almost equivalent test accuracies of 0.97 for the original model and 0.96 for the additional LSTM layer model. There also does not seem to be stark differences in the errors both models make (Table 2), highlighting that an additional LSTM layer might not be needed for this model. One LSTM layer is sufficient enough to capture temporal dependencies.

Table 2: Text-to-Text prediction comparison on additional LSTM layer

Equation	**Prediction (Base)**	**Prediction (Add. LSTM)**	**Correct Answer**
53 + 25	82	86	78
10 - 20	-1	-1	-10
43 + 34	86	76	77
5 + 57	62	66	62
69 + 51	121	122	120
59 - 81	-26	-25	-22
55 + 36	93	94	91
15 + 1	23	13	16
98 - 85	10	16	13
40 - 57	-13	-15	-17
17 - 62	-40	-44	-45

Image-to-Text Addition of an extra LSTM encoder layer for the Image-to-Text model does seem to have its benefits, boosting its test accuracy on the 0.2 train-test split towards 0.93, previously 0.90. This could be because the increased model complexity aligns well with increased input complexity. Where the original one LSTM encoder layer in the one-hot coded text input was sufficient, now multiple LSTM layers might be needed to extract the temporal patterns in the sequence, resulting in a higher test accuracy (Figure 11).

(a) Accuracy and loss on base model



(b) Accuracy and loss on model with additional LSTM layer

Figure 10: Comparison between loss over training splits

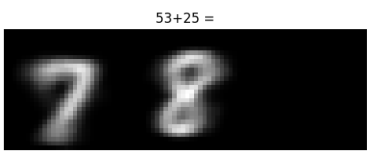
Text-to-Image Once again direct improvements are hard to quantify on the Text-to-Image model. We can make a direct comparison between the loss values per training set on both models (Figure 11). However, purely based on loss the models seem quite comparable. The additional LSTM layer model achieves the lowest loss value of just below 0.158, which only differs on 0.002 with the original model. This difference can also not be seen in the clarity of the images (Figure 12), further reinforcing that the fuzziness of the images stem from the loss function rather than the latent space not encapsulating enough information. This leads to the conclusion that an extra encoder layer is not needed for the Text-to-Image model. The comparable results with the Text-to-Text model make sense as in both cases the conclusion could already be drawn that the encoder captured enough information. The discrepancy with the Image-to-Text model is then also logical, as the input now becomes more complex insinuating the need for more complex encoders.

(a) Loss on base model

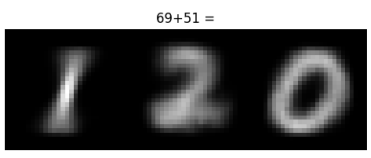
(b) Loss on model with additional LSTM layer

Figure 11: Comparison between loss over training splits

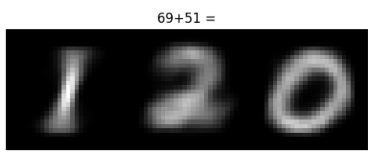
(a) base pred



(b) add. LSTM pred



(c) base pred



(d) add. LSTM pred

Figure 12: Image prediction of equations

References

Géron, A. (2022). *Hands-on machine learning with scikit-learn, keras, and tensorflow*. O'Reilly Media, Inc.

Interpolation and traversal in latent space. (2025). https://apxml.com/courses/autoencoders-representation-learning/chapter-6-latent-space-manipulation/latent-space-interpolation-traversal

Kingma, D. P., & Welling, M. (2013). Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*.

Larsen, A. B. L., Sønderby, S. K., Larochelle, H., & Winther, O. (2016, 20–22 Jun). Autoencoding beyond pixels using a learned similarity metric. In M. F. Balcan & K. Q. Weinberger (Eds.), *Proceedings of the 33rd international conference on machine learning* (pp. 1558–1566, Vol. 48). PMLR. https://proceedings.mlr.press/v48/larsen16.html

Pelt, D. (2025). *Introduction to deep learning session 10: Unsupervised learning and gans*. https://brightspace.universiteitleiden.nl/d2l/le/lessons/388351/topics/3729052

Ramadhan, L. (2021). *Neural network: The dead neuron*. https://towardsdatascience.com/neural-network-the-dead-neuron-eaa92e575748/